

1. Introduction

This project consists of a simulation model of a restaurant. The main objective is to understand the relationship among the various actors and related processes (table assignment, ordering, eating, etc.), to identify and measure inefficiencies and use this information to optimize the relevant parameters (number of employees, menu items, etc.) and the required objectives (improve waiting times, reduce employees overtime, etc.). The simulation spans one week, from monday to sunday, and is time aware: some parameters change according to the time of day and day of the week (customers arrival rate, employees schedules, etc.).

2. Model

To model the complexities of this scenario the selected simulation patterns were discrete events, used as a scaffold to model the process flow, and agent based, used to model the complex behavior of the agents and the interactions among them.

The process is orchestrated by the PML model that triggers the arrival of customers and manages their flow through the system, manages the resources (waiters and cooks) through Seize-Service-Release and assigns tasks using messages. Task-specific operations, such as the arrival of a customer and subsequent table assignation, are managed by the agents' internal statecharts, which exchange information through messages.

The actors involved in this process are the following:

- **CustomerGroup**: a group of customers who share the same table, modeled as a single entity for simplicity;
- **Waiter**: the main resource of the simulation. Its duties include welcoming customers, assigning tables, taking orders, delivering them to the cooks and back, and managing checkout operations;
- **Cook**: its main duty is to take orders, prepare them and give them back to the waiters;
- **Table**: a resource that has different availability based on the time of day and location;
- **Menu**: contains the different dishes (**Menu Items**) available during different parts of the day. The joint selection of categories of its elements is described by correlations and marginal probabilities;
- **Order**: stores the Menu Items selected by a Customer Group.

The process is as follows. First, a customer group arrives at the entrance of the restaurant. A waiter comes to welcome the group and assigns a table to it, according to the current availability. If there are no tables available, the group steps aside and waits until a table of suitable size is freed; or if there are no tables big enough, gives up and leaves.

Once the group arrives at the table it reads the menu. Every customer chooses some dishes and when everyone is ready a waiter is called. The waiter takes the order and brings it to the kitchen where it's placed in a FIFO queue. When a cook becomes available he takes the order, prepares it and places it in another FIFO queue. When a waiter becomes available, he takes it to the corresponding table.

After the group finishes the meal, it goes to the register, waits for a waiter to handle the payment, pays, and then leaves the restaurant.

A central part of this simulation is that it takes into account that the flow and behavior of customers varies according to the time and day of the week. For example, in the mornings the groups tend to be smaller and spend less time at the table, or, in the weekends people tend to be in bigger groups and spend more time in the restaurant.

To model this behavior the days were divided in five parts: BREAKFAST (7:00 - 11:00), LUNCH (11:00 - 15:00), BREAK (15:00 - 19:00), DINNER (19:00 - 23:00), CLOSED (23:00 - 7:00). And then, for each time varying quantity, some tables were defined specifying the required parameters for each time period.

In addition to the groups' flow, size and eating habits, other time-varying parameters are included: the menu offers different options for different parts of the day; the availability of the employees varies according to a schedule (and also does their hourly compensation); different groups of tables are available at different times and with different priorities.

One important distinction between customer behavior and employees/resources availability is that the former is an observed phenomenon, whereas the latter can be optimized. For this reason the first parameters should be estimated from measurements taken in the real world (their main purpose is to make the model as close to reality as possible or to simulate what-if scenarios), while the second parameters can be varied experimentally, and their values should be selected manually (or through optimization based on the customer behavior values).

2.1 Customers

To model the behavior of the customers the following random quantities are used:

- 2.1.1 Arrival rate:** the generation of CustomerGroup follows a nonhomogeneous Poisson process whose intensity $\lambda(t)$ varies according to the weekday and the hour. This stochastic process was chosen because it effectively models the randomness of arrivals specifying only the average arrival rate. The non homogeneity was necessary to represent the daily and weekly peaks that "naturally" occur in restaurant demand. The different $\lambda(t)$ (arrival rates) are defined in a table (fig. 1).

start_timestamp	end_timestamp	arrival_rate
1970-01-05 07:00:00	1970-01-05 08:00:00	5
1970-01-05 08:00:00	1970-01-05 09:00:00	12
1970-01-05 09:00:00	1970-01-05 10:00:00	14
1970-01-05 10:00:00	1970-01-05 11:00:00	8
1970-01-05 11:00:00	1970-01-05 12:00:00	6
1970-01-05 12:00:00	1970-01-05 13:00:00	18
1970-01-05 13:00:00	1970-01-05 14:00:00	28
1970-01-05 14:00:00	1970-01-05 15:00:00	16
1970-01-05 15:00:00	1970-01-05 16:00:00	4
1970-01-05 16:00:00	1970-01-05 17:00:00	3
1970-01-05 17:00:00	1970-01-05 18:00:00	5
1970-01-05 18:00:00	1970-01-05 19:00:00	12
1970-01-05 19:00:00	1970-01-05 20:00:00	18
1970-01-05 20:00:00	1970-01-05 21:00:00	20
1970-01-05 21:00:00	1970-01-05 22:00:00	12
1970-01-05 22:00:00	1970-01-05 23:00:00	5
1970-01-05 23:00:00	1970-01-06 07:00:00	0
1970-01-06 07:00:00	1970-01-06 08:00:00	5
1970-01-06 08:00:00	1970-01-06 09:00:00	14
1970-01-06 09:00:00	1970-01-06 10:00:00	15
1970-01-06 10:00:00	1970-01-06 11:00:00	8
1970-01-06 11:00:00	1970-01-06 12:00:00	6
1970-01-06 12:00:00	1970-01-06 13:00:00	19
1970-01-06 13:00:00	1970-01-06 14:00:00	29
1970-01-06 14:00:00	1970-01-06 15:00:00	16

(fig. 1: A snippet of a sample table (arrival_rate_week_groupsize). The column arrival_rate contains the value $\lambda(t)$. Every row specifies $\lambda(t)$ when t is between start_timestamp and end_timestamp)

2.1.2 Group size: values are sampled from a truncated normal distribution (cast to integer) with a minimum value 1, maximum value 12, a time-dependent mean, and a standard deviation of 1. This distribution was chosen because it allows the group size to be centered around a typical value preventing negative or 0 values.

The time-dependent mean values are defined in the table `arrival_rate_week_groupsize` (fig. 2).

start_timestamp	end_timestamp	avg_group_size
1970-01-05 07:00:00	1970-01-05 08:00:00	1,3
1970-01-05 08:00:00	1970-01-05 09:00:00	1,3
1970-01-05 09:00:00	1970-01-05 10:00:00	1,3
1970-01-05 10:00:00	1970-01-05 11:00:00	1,3
1970-01-05 11:00:00	1970-01-05 12:00:00	2,2
1970-01-05 12:00:00	1970-01-05 13:00:00	2,2
1970-01-05 13:00:00	1970-01-05 14:00:00	2,2
1970-01-05 14:00:00	1970-01-05 15:00:00	2,2
1970-01-05 15:00:00	1970-01-05 16:00:00	1,8
1970-01-05 16:00:00	1970-01-05 17:00:00	1,8
1970-01-05 17:00:00	1970-01-05 18:00:00	1,8
1970-01-05 18:00:00	1970-01-05 19:00:00	1,8
1970-01-05 19:00:00	1970-01-05 20:00:00	2,8
1970-01-05 20:00:00	1970-01-05 21:00:00	2,8
1970-01-05 21:00:00	1970-01-05 22:00:00	2,8
1970-01-05 22:00:00	1970-01-05 23:00:00	2,8
1970-01-05 23:00:00	1970-01-06 07:00:00	2
1970-01-06 07:00:00	1970-01-06 08:00:00	1,3
1970-01-06 08:00:00	1970-01-06 09:00:00	1,3
1970-01-06 09:00:00	1970-01-06 10:00:00	1,3
1970-01-06 10:00:00	1970-01-06 11:00:00	1,3
1970-01-06 11:00:00	1970-01-06 12:00:00	2,2
1970-01-06 12:00:00	1970-01-06 13:00:00	2,2
1970-01-06 13:00:00	1970-01-06 14:00:00	2,2
1970-01-06 14:00:00	1970-01-06 15:00:00	2,2

(fig. 2: A snippet of a sample table (`arrival_rate_week_groupsize`). The column `avg_group_size` contains the value $\mu(t)$. Every row specifies $\mu(t)$ when t is between `start_timestamp` and `end_timestamp`)

2.1.3 Eating time: the eating time is computed using a random base value corrected according to the group size.

The computation starts with a random draw from a lognormal random variable. Its mean is time dependent (*eatingTime*), its standard deviation is equal to 20% of the mean, and it has a min value of 5 minutes. The time-dependent mean eating times are defined in an external table (fig. 3).

This random variable was chosen because it enforces a bounded minimum (customers cannot eat too quickly) and has a long right tail, allowing in rare cases the possibility of very long eating times (customers that spend a lot of time at the table). The standard deviation is set as a percentage of the mean to introduce variability that scales proportionally with the expected eating time.

This random value (*baseMeanTime*) is then corrected with an exponential saturation model to take into account for the group size:

$$finalEatingTime = baseMeanTime \cdot \left(1 + maxIncrease \cdot (1 - e^{\{-growth \cdot (groupSize - 1)\}})\right)$$

The parameter *maxIncrease* (0.8) and *growth* (0.35) determine the maximum possible increase and the rate at which saturation is reached as the group size grows.

This choice is consistent with the modeling assumption that larger groups spend more time at the table talking, however that growth is not linear since the impact of each additional

element of the group decreases progressively and meal duration cannot increase indefinitely.

start_timestamp	end_timestamp	period	time_avg
05/01/1970 07:00	05/01/1970 11:00	breakfast	5
05/01/1970 11:00	05/01/1970 15:00	lunch	20
05/01/1970 15:00	05/01/1970 19:00	break	10
05/01/1970 19:00	05/01/1970 23:00	dinner	40
05/01/1970 23:00	06/01/1970 07:00	closed	0

(fig. 3: a sample table (eating_time) containing average eating times (time_avg) for every period of the day for a single customer.)

2.1.4 Menu reading and order placement: the time spent reading the menu and communicating the order to the waiter is computed using a random base time, adjusted according to the group size.

The base time spent is obtained through a random draw from a truncated normal random variable with a minimum value of 0 minutes, an unbounded maximum, a mean of 6 minutes and a standard deviation of 2 minutes. This distribution is used to introduce stochastic variability and to prevent negative values.

This base time is then corrected through this equation:

$$totalTime = baseTime * (1 + k * (groupSize - 1))$$

Where the parameter k (0.05) represents the additional percentage increase in time introduced by each additional component of the group. For example a sampled time of 6 minutes becomes 6.3 minutes for a group of two and 6.6 minutes for a group of three.

2.1.5 Pay time: as a simplification, the time required for payment depends on whether one person pays for the whole group or each member pays individually.

The time spent by a single member is obtained through a random draw from a truncated normal random variable with a minimum value of 5 seconds, an unbounded maximum, a mean value of 45 seconds and standard deviation of 10 seconds.

Assuming that the draws are independent, the properties of the sum of normal random variables can be used to model the case in which each member of the group pays separately. A draw from a sum of IID normal random variables $X_i \sim N(\mu, \sigma^2)$ can be made creating a new variable, multiplying the mean and the variance by the number of components of the group (n). The mean becomes $mean * n$ and variance $n\sigma^2$, so the standard deviation becomes $\sqrt{n * \sigma^2} = \sigma\sqrt{n}$. The total pay time can be drawn from: $X_i \sim N(n\mu, \sqrt{n}\sigma)$.

To choose between these two scenarios a random draw from a Bernoulli random variable is performed with parameter $p = 0.7$ set to favor the case for which one person pays for the whole group. Then, to sample the time this formula is sufficient:

$$(onePaysAll) * (normal(min, max, \mu, \sigma)) + (1 - onePaysAll) * (normal(min, max, \mu * n, \sigma\sqrt{n}))$$

2.2 Employees and resources

Employee and resource availability are also time-dependent. For this reason, the following tables and random variables are defined:

2.2.1 Waiters: a schedule describes how the number of available waiters varies for each hour of every day (fig. 4).

start_timestamp ▾	end_timestamp ▾	waiters ▾
05/01/1970 07:00	05/01/1970 11:00	2
05/01/1970 11:00	05/01/1970 15:00	5
05/01/1970 15:00	05/01/1970 19:00	3
05/01/1970 19:00	05/01/1970 23:00	6
05/01/1970 23:00	06/01/1970 07:00	0
06/01/1970 07:00	06/01/1970 11:00	2
06/01/1970 11:00	06/01/1970 15:00	5
06/01/1970 15:00	06/01/1970 19:00	3
06/01/1970 19:00	06/01/1970 23:00	6
06/01/1970 23:00	07/01/1970 07:00	0

(fig. 4: part of a sample waiter availability table)

2.2.2 Cooks: a schedule describes how the number of available cooks varies for each hour of every day (fig. 5).

start_timestamp ▾	end_timestamp ▾	cooks ▾
05/01/1970 07:00	05/01/1970 11:00	2
05/01/1970 11:00	05/01/1970 15:00	4
05/01/1970 15:00	05/01/1970 19:00	2
05/01/1970 19:00	05/01/1970 23:00	5
05/01/1970 23:00	06/01/1970 07:00	0
06/01/1970 07:00	06/01/1970 11:00	2
06/01/1970 11:00	06/01/1970 15:00	4
06/01/1970 15:00	06/01/1970 19:00	2
06/01/1970 19:00	06/01/1970 23:00	5
06/01/1970 23:00	07/01/1970 07:00	0

(fig. 5: part of a sample cook availability table)

The **time to cook** is computed as a stochastic value: for every item of the menu a cooking time mean value and standard deviation are defined (as explained in 2.3). Using these two values the final preparation time is sampled from a truncated normal random variable with minimum time 0 (precooked), an unlimited maximum and the associated mean and standard deviation.

For simplicity, to compute the time required to prepare orders that contain multiple items, the single sampled times of each item are summed.

2.2.3 Tables: a schedule indicates, for each day part, which tables are available to use and with which priority (i.e. during lunch both inside and outside tables can be used, but inside tables are preferred. During breakfast only the bar tables are available, etc.).

The tables are divided into three spaces: bar, inside and outside. The number and capacities of the tables are parameters of the model (for simplicity the number of tables and their respective capacities don't change relative to the time and day of week).

start_timestamp	end_timestamp	bar_on	inside_on	outside_on	bar_prio	inside_prio	outside_prio
05/01/1970 07:00	05/01/1970 11:00	VERO	FALSO	FALSO	1	2	3
05/01/1970 11:00	05/01/1970 15:00	FALSO	VERO	VERO	3	1	2
05/01/1970 15:00	05/01/1970 19:00	VERO	VERO	FALSO	1	2	3
05/01/1970 19:00	05/01/1970 23:00	FALSO	VERO	VERO	3	1	2
05/01/1970 23:00	06/01/1970 07:00	FALSO	FALSO	FALSO	1	2	3

(fig. 6: a sample table availability and priority table)

2.3 Menu

The menu is an important part of the simulation: it drives profits, determines servicing times and introduces additional stochastic behavior.

For each period of day the food items available vary. Each item has:

- **Popularity** (popularity_weight): an integer describing how many times the food item was chosen based on observations from real life or predictions. The empirical probability of a food item can be computed taking its observed count (w_i) and dividing it by the sum of the observed counts of the dishes in the choice set ($\sum_j w_j$): $P(i) = \frac{w_i}{\sum_j w_j}$;
- **Preparation time parameters:** mean time and standard deviation of the preparation time, computed from observed values or predictions (usage explained in 2.2.2);
- **Price;**
- **Category:** the category of the item, which can be one of the following: DRINK, MAIN, SIDE, DESSERT, COMBO, FIXED_MENU;
- **Course:** the role of the food item in the meal, the associations are the following:
 - MAIN dishes can be either a "1st" or a "2nd" course;
 - DRINK is a "0th" course;
 - SIDE is a "1st" course;
 - DESSERT is a "3rd" course;
 - COMBO and FIXED_MENU are "4th" courses;

name	dayPart	category	course	popularityWeight	price	prepTimeMeanMin	prepTimeSdMin
Espresso	BREAKFAST	DRINK	0	30	1,3	1	0,3
Cappuccino	BREAKFAST	DRINK	0	24	1,8	2	0,5
Tea	BREAKFAST	DRINK	0	8	2	2,5	0,6
Plain croissant	BREAKFAST	MAIN	1	20	1,5	0,8	0,2
Filled croissant	BREAKFAST	MAIN	1	14	2,2	1,2	0,3
Ham and cheese toast	BREAKFAST	MAIN	1	10	4,5	4	1
Breakfast combo coffee + croissant + juice	BREAKFAST	COMBO	4	18	4,8	2,5	0,6
Orange juice	BREAKFAST	DRINK	0	9	2,5	1	0,2

(fig. 7: a sample menu for BREAKFAST)

2.3.1 Copulas

As we have already seen in the previous section, each item has an empirical marginal probability defined on the basis of its observed popularity. This value describes how likely each dish is to be selected individually. However, this information alone is not sufficient to produce realistic orders, because it does not take into account the fact that the choice of a given category of dishes conditions the subsequent choice of other dishes. For example if someone orders a fixed menu it's unlikely that they will order something else, or if someone orders a second course they are more likely to order also a side dish than someone that ordered a first course.

To solve this problem a copula-based approach was used to introduce the dependency structure. For this purpose, for each period of day (BREAKFAST, LUNCH, etc.), a correlation matrix was defined among all the categories available in that period (fig. 8), together with a table containing the marginal probabilities for the choice of each category (fig. 9).

variable	fixed_menu	course1	course2	side	drink	dessert
fixed_menu	1	-0,6	-0,55	-0,35	-0,25	-0,2
course1	-0,6	1	0,25	0,15	0,2	0,1
course2	-0,55	0,25	1	0,3	0,15	0,1
side	-0,35	0,15	0,3	1	0,08	0,1
drink	-0,25	0,2	0,15	0,08	1	0,05
dessert	-0,2	0,1	0,1	0,1	0,05	1

(fig. 8: correlation matrix for LUNCH)

dayPart	variable	marginal_probability
BREAKFAST	main	0,75
BREAKFAST	drink	0,8
BREAKFAST	combo	0,18
BREAK	main	0,7
BREAK	drink	0,65
LUNCH	fixed_menu	0,22
LUNCH	course1	0,75
LUNCH	course2	0,45
LUNCH	side	0,35
LUNCH	drink	0,55
LUNCH	dessert	0,3
DINNER	fixed_menu	0,12
DINNER	course1	0,55
DINNER	course2	0,65
DINNER	side	0,4
DINNER	drink	0,6
DINNER	dessert	0,38

(fig.9: marginal probabilities for each category)

2.4 Order

As we have seen, the choice of dishes to include in an order is conditioned by their popularity and the dependency structure introduced by the copula.

By exploiting the copula, a sample coherent with the day part in which the order is placed is drawn. The sample indicates which categories are selected simultaneously. Finally, for each included category, one food item is drawn according to its popularity and inserted into the order.

The process is as follows: first of all, the correlation matrices and the marginal probabilities associated with the current day part are gathered (it's essential that the variables of the two tables match). For example, in the BREAKFAST scenario, we'll have the tables in fig.10 and fig. 11.

dayPart	variable	marginal_probability
BREAKFAST	main	0,75
BREAKFAST	drink	0,8
BREAKFAST	combo	0,18

(fig. 10: marginal probabilities for BREAKFAST)

variable	main	drink	combo
main	1	0,35	-0,7
drink	0,35	1	-0,55
combo	-0,7	-0,55	1

(fig. 11: correlation matrix for BREAKFAST)

Once the correlation matrix (m) is obtained we use the Cholesky decomposition to obtain the following factorization: $m = LL^T$, where L is a lower triangular matrix that allows us to transform a vector of independent standard normal variables into a vector of correlated standard normal variables according to the specified correlation matrix (m).

At this point we generate a vector of values sampled from a standard normal variable, with a component per variable, and introduce the dependence multiplying L by the vector.

Having obtained the correlated draws we need to determine which categories will be inserted into the order. For this purpose we compute some thresholds using the marginal probabilities and the inverse cumulative probability function of the standard normal distribution.

Given that we want the probability of the category being over the threshold to be equal to its marginal probability (p_i) we define the threshold (t) accordingly:

$$\begin{aligned}P(Z_i > t_i) &= p_i \rightarrow \\P(Z_i \leq t_i) &= 1 - p_i \rightarrow \\t_i &= \Phi^{-1}(1 - p_i)\end{aligned}$$

This way we'll have that if p_i is high then t_i will be low and vice versa. Once the threshold are obtained we select only the categories with a value sufficiently large.

At last, for each selected category, we take all the dishes available for that category and using their popularity we perform a weighted draw and insert into the order the result.

The weighted extraction is performed as follows: the popularities of all the candidates get summed and a number between 0 and the sum is drawn from a uniform distribution. Then, the candidates are considered one by one, each adding its popularity to a cumulative sum. The first candidate whose weight makes the cumulative sum get greater than the random draw gets selected.

The process is repeated for every element of the customer group, and it's ensured that the order contains at least one element of the menu per person.

2.5 Additional assumptions and simplifications

- There are no "reservations". The flow of customers is regulated only by the couple time-day and the related rate of arrival.
- All cooks can work in parallel and a single cook prepares the whole order.
- There cannot be any customer complaints (dished cannot be remade).
- Customers never get tired of waiting (they can be discarded only if they are too big or if they are still in queue when the restaurant closes).
- Employees can never take a sick day (it's better to experiment with understaffed scenarios editing the timetables to have more control)
- The tasks are not prioritized and the employees accept whichever task is requested first.
- Tables cannot be joined or groups separated. If a group does not fit the biggest table it can't enter the restaurant.

3. Implementation

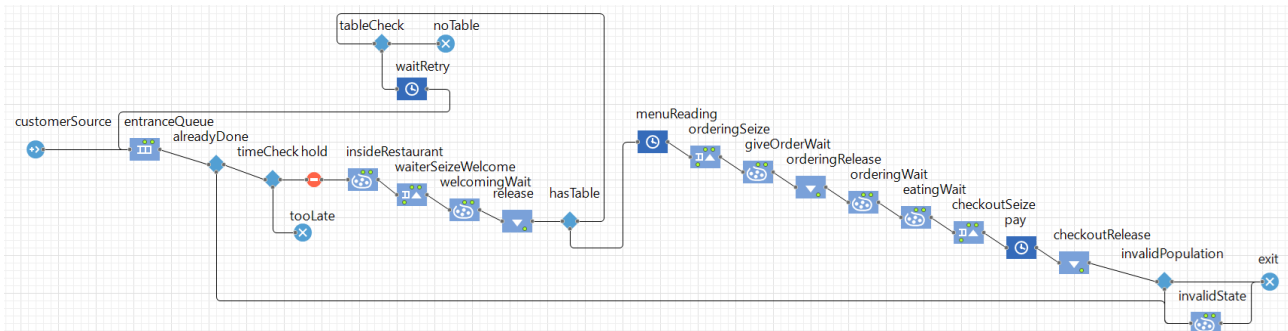
The model for this project was implemented in AnyLogic through the combined use of two paradigms: Process Modeling Library (PML) and Agent Based Modeling.

The PML serves as a scaffold for the customers flow. It's used to generate and manage resources, guide customers through the process, and assign tasks to employees.

The Agents have different capabilities and exchange information through messages. Every type of agent has an internal statechart, that describes it's behavior.

The schedules rely on the database built in AnyLogic. The parameter tables are defined in excel and imported. The xlsx file containing the tables can be found in `scripts/restaurant_variables.xlsx`.

3.1 The customers flow



(fig. 12: the customers PML flow)

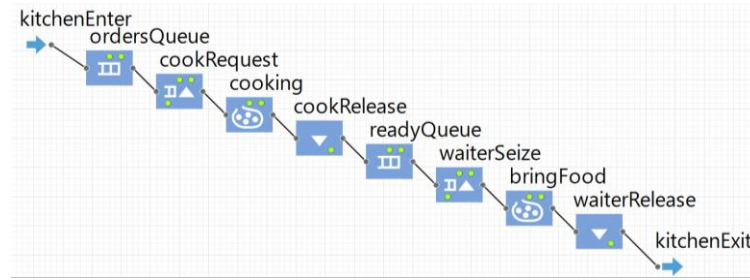
The customers flow is governed by the PML model (fig. 12) and the internal state of the CustomerGroup agent. It's important that the agent's internal state is always synchronized with the PML flow. This synchronization is achieved through the use of waits in the PML, and through messages in the agent's internal statechart.

The flow starts with the generation of CustomerGroups, with arrival rate defined by the ArrivalsSchedule schedule. The CustomerGroups wait in the entranceQueue (a priority queue) and, one at a time, enter the table assignation loop. If the restaurant is still open, a waiter is seized and assigns a table to the group. If there are no tables of suitable size available, the group is placed back into the entranceQueue with a higher priority and a 5 minute delay, to allow smaller groups to enter the restaurant (or, if the group is bigger than the biggest table, it leaves the restaurant). Once a customerGroup receives a message with a Table from the waiter, it reaches the assigned table and starts reading the menu (as explained in 2.1.4), and creates an order (as explained in 2.4).

Then the order exchange process begins. A waiter is seized, and the customerGrups waits into the GiveOrderWait block. The waiter reaches the group and sends its instance to it. The group then sends the Order to the Waiter, that takes it to the kitchen, injects it into the kitchen PML flow, frees the CustomerGroup from the GiveOrderWait, and gets freed.

The CustomerGrup then enters the orderingWait, from which it is released when a waiter sends a message notifying that the order is ready. While the group eats the food (according to 2.1.3) it remains in the EatingWait block. Once it finishes, it walks to the exit node, frees itself from the wait, releases the table, and a waiter is seized to handle the payment. The group pays (according to 2.1.5), the waiter gets released, and the customerGroup exits the restaurant.

3.2 The kitchen flow

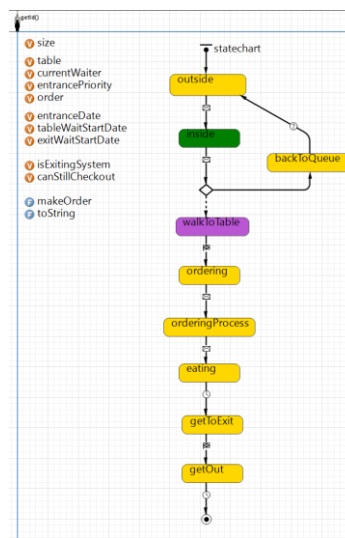


(fig. 13: the kitchen PML flow)

The flow of orders through the kitchen is managed by the kitchen PML model (fig. 13).

Orders are sent by the CustomerGroups and received by the waiters who, in turn, inject them into the kitchen flow. The orders are placed into a FIFO queue and, once a cook is seized, processed. When the preparation is completed (according to 2.2.2), the cook frees the order and gets released. The cooked orders are then placed into another FIFO queue, where a waiter is seized to deliver the order to the corresponding customerGroup. After that, the waiter releases the order from the wait block, gets released, and the order exits the kitchen flow (an exit block is used because the Order must not be destroyed when leaving the flow).

3.3 CustomerGroups



(fig. 14: the internal statechart of a CustomerGroup)

The behavior of customer groups is governed by their internal statechart (fig. 14).

Once the CustomerGroup enters the restaurant, it transitions to the "outside" state. When it enters the table assignment loop it receives a "welcome" message, and transitions to the "inside" state. Here it waits for a message of type Table. If the received Table is an instance of NoTable (NoTable is a subclass of Table) the CustomerGroup is placed back into the queue. Otherwise, it transitions to the "walkToTable" state.

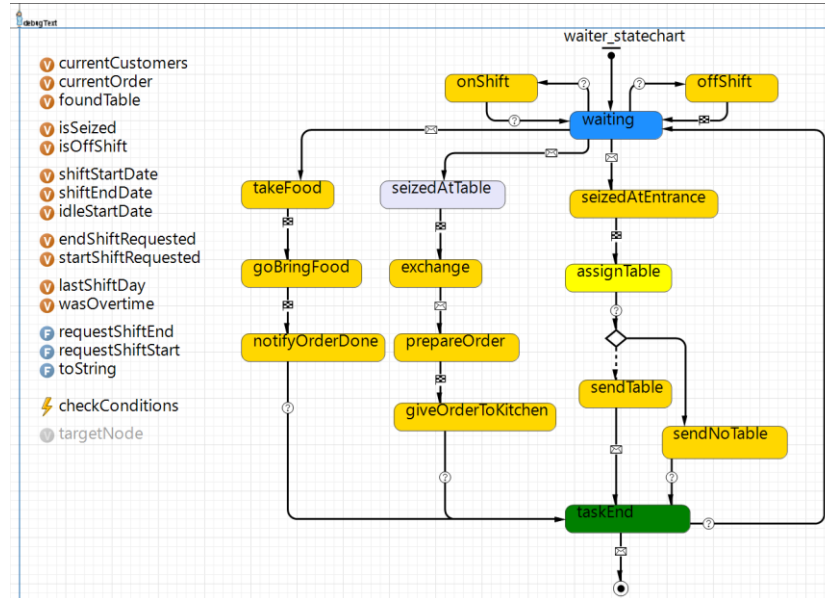
Upon reaching the assigned table, the CustomerGroup notifies the Waiter by sending a "customersArrivedAtTable" message. After receiving the instance of a new Waiter it enters the "orderingProcess" state where an order is created and sent to the Waiter.

The CustomerGroup waits for a "foodReady" message, and then remains in the "eating" state for an amount of time defined by the eatingTimeModel (according to 2.1.3). Finally, it enters the "getToExit"

state and, upon reaching the node, transitions to the "getOut" state where it remains until the PML flow destroys the agent.

CustomerGroups belong in the "customerGroup" population in the Main agent.

3.4 Waiters



(fig. 15: the internal statechart of a Waiter)

The behavior of the waiters is governed by their internal statechart (fig. 15).

The main state of the agent is "waiting". In this state, the Waiter waits for a message indicating which behavior branch to execute and resets dirty variables used by previous operations.

Three behavior branches can be reached, each triggered by a different type of message:

- The "goWelcome" message triggers the customer group welcoming and table assignment behavior: when seized in the waiterSeizeWelcome block, the waiter approaches the entranceNode and searches for a free table of suitable size for the customerGroup waiting in the entrance. If a suitable table is found, it is sent to the customerGroup and the waiter moves to the table. Otherwise, the Waiter sends a NoTable message. The customerGroup notifies the Waiter when it reaches the assigned table.
- A goServe message (containing a CustomerGroup) triggers the order taking behavior: When seized in the orderingSeize block, the Waiter approaches the customerGroup and sends its own instance to it. Once the Waiter receives the Order, it approaches the kitchen and injects the order into the kitchen PLM flow.
- A goBringFood message (containing an Order) triggers the order delivery behavior: When seized in the waiterSeizeReady block, the Waiter approaches the kitchen, picks up the Order, brings it to the correct customerGroup and sends a "foodReady" message to it.

Waiters belong in the "waiters" population in the Main agent.

3.4.1 Shifts

A waiter can be either on or off shift. Shift management is achieved through the `waitersSchedule`, the `waitersPool` and the agent's internal statechart.

A significant effort was made to prevent a waiter from becoming unavailable while still executing a process. After extensive testing it became apparent that the tools provided by the `resourcePool` were not enough: when a schedule is used to define the pool's capacity, resource availability changes immediately according to the schedule, with no flexibility, even if the resources that are being made unavailable are currently seized. This leads to the freeze of the seizing `customerGroups` and the subsequent unexpected halt of the PML flow until the resource is made available again.

To allow waiters to complete their task before ending their shift, a custom shift management system was implemented.

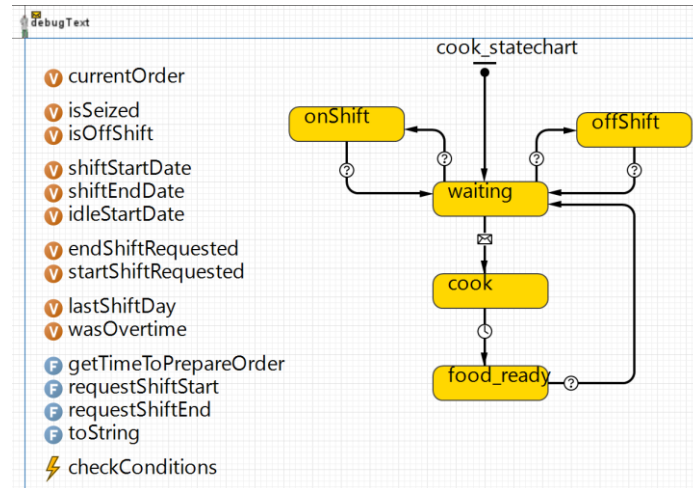
Waiters can be seized only through the `waitersPool`, which is initialized with the maximum possible number of waiters and allows resources to be seized only if they are currently on shift and don't have any other running task. Whenever the schedule changes, the required number of waiters is computed and, if there are fewer waiters on shift than required an appropriate number of waiters receive a `requestShiftStart()` call. Else, if there are too many waiters on shift, an appropriate number of waiters receive a `requestShiftEnd()` call.

The functions `requestShiftStart()` and `requestShiftEnd()` control the boolean flag `isOffShift`, which determines whether a waiter is available to be seized. After one of these functions is called, as soon as the conditions are met and the waiter returns to the "waiting" state, the corresponding state ("onShift" or "offShift") is reached. These states update the flag, register the event, and update the necessary information to prepare the waiter for the next state change.

To account for overtimes, the `offShift` state can also be reached after the restaurant has closed and no customers remain inside. This mechanism is also used to reliably register overtimes: if a waiter reaches the `offShift` state but `requestShiftEnd()` was not called, it means that it was serving customers after the restaurant had closed.

It's important to note that a resource agent should not be interpreted as an individual employee, but as a slot of availability. It's not unusual for a single "waiter" to appear to work extremely long shifts. It's an abstraction: a "Waiter" represents a unit of availability, regardless of which individual fulfills it.

3.5 Cooks



(fig. 16: the internal statechart of a Cook)

The behavior of the cooks is governed by their internal statechart (fig. 16).

The main state of the agent, similarly to the waiters, is "waiting". In this state, the cook waits for a goCook(Order) message (sent by the seize block in the kitchen flow as seen in 3.2), which starts the cooking process. The cookingProcess is simplified as a single timeout (computed as explained in 2.2.2). Once the timeout expires, the order gets freed in the kitchen flow and proceeds to the next steps. Once the operation is completed, the Cook returns to the "waiting" state.

Shifts management is analogous to that of the waiters (3.4.1). In this case, it's achieved through the cooksSchedule, the cooksPool and the cooks's internal statechart.

Cooks belong to the "cooks" population in the main.

3.6 Tables

Tables are modeled as Agents and used as resources. They don't have any internal behavior and are managed manually. The resource pools are used only to simplify the positioning of Tables in the appropriate nodes and attractors, since their time and priority based selection logic is too complex and needed to be implemented manually.

The table selection logic is described in main.tablesSelection and is used exclusively by the Waiters during the goWelcome behavior (3.4). Using the schedules (explained in 2.2.3), it selects the first available and unoccupied table with the highest associated priority.

The tables have two variables used during the selection process:

- isFree: a boolean indicating whether the table is currently occupied by a CustomerGroup. To avoid inconsistencies, it's only edited by the main.tableSelection function during table assignment and by the CustomerGroup when leaving the restaurant;
- waiterAssinger: stores a reference to the waiter that assigned the table, used by the customerGroup to communicate when they have reached the table.

Tables are divided according to their location (bar, outside, inside) and are stored in separate populations in the main (batTables, outsideTables, insideTables).

3.7 Order

Orders are modeled as Agents. Each order stores a reference to its associated CustomerGroup, the selected MenuItem, a timestamp, and a function that computes the its total price.

Orders belong to the "orders" population in the main.

3.8 Menu

The menu is modeled through four classes, each responsible for a different aspect of the simulation:

- MenuItem: represents a single menu item;
- MenuItem: loads the menu items from the database, groups them by daypart, and generates orders;
- MenuCopulaGenerator: loads the copulas from the database and generates samples consistent with them;
- MenuCopulaUpdater: an utility class that computes new copulas from a list of existing orders.

The menu contents and the correlation matrices rely on the database built in AnyLogic. The parameter tables are defined in excel and imported. The .xlsx file containing the tables can be found in scripts/menu.xlsx.

3.8.1 MenuItem

This class represents a single item of the menu (as explained in 2.3).

Each item belongs to one of the following categories:

- DRINK;
- MAIN;
- COMBO;
- SIDE;
- DESSERT;
- FIXED_MENU;

For simplicity, the list of categories is hardcoded.

3.8.2 MenuItem

This class loads the menuItems from the database, groups items by dayPart and generates Orders using samples produced by MenuCopulaGenerator and the popularity weights of the available items.

Orders are generated as described in section 2.4.

For simplicity, this class provides only functions to generate orders for the dayParts described in the section 2 and supports only copulas with the categories described in section 3.8.1.

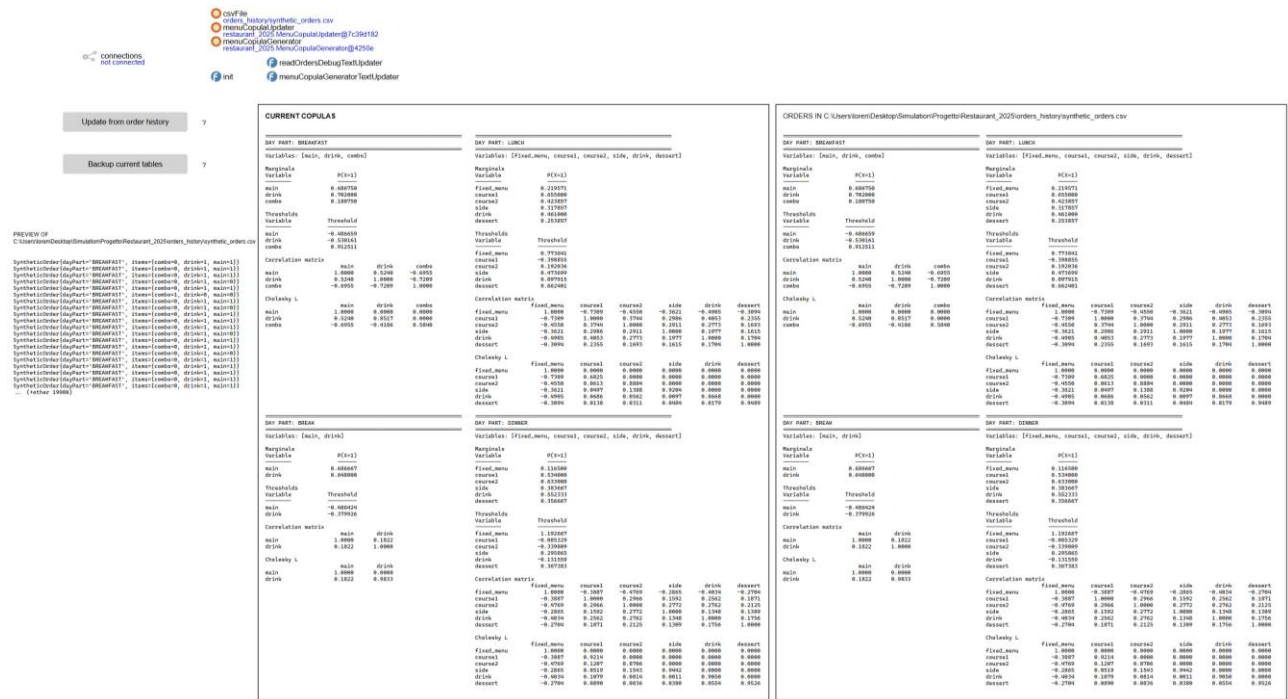
Here is a list of the supported dayParts/categories:

BREAKFAST	-> "main", "drink", "combo"
BREAK	-> "main", "drink"
LUNCH	-> "fixed_menu", "course1", "course2", "side", "drink", "dessert"
DINNER	-> "fixed_menu", "course1", "course2", "side", "drink", "dessert"

3.8.3 MenuCopulaGenerator

This class loads the copulas for BREAKFAST, LUNCH, BREAK and DINNER from the database, and provides functions to generate samples consistent with the corresponding copulas (as described in 2.4). The generated samples are arrays of CopulaSample objects, which provide methods to determine which variables have been drawn.

3.8.4 MenuCopulaUpdater



(fig. 17: the interface for the MenuCopulaUpdater)

This is a utility class that can be used to compute copulas from an existing CSV file of real orders. Its functionality can be accessed through the ConfigurationUpdaterSimulation simulation (fig. 17).

For simplicity the available variables are hardcoded (in the same way as MenuItems in section 3.8.2).

An example CSV file is provided in orders_history/synthetic_orders.csv and is has the following format:

```
order_id,dayPart,main,drink,combo,fixed_menu,course1,course2,side,drink,dessert
BRE_00001,BREAKFAST,1,1,0,,,,,
BRE_00002,BREAKFAST,1,1,0,,,,,
BRE_00003,BREAKFAST,1,1,0,,,,,
...
BRA_00001,BREAK,1,1,,,,,
BRA_00002,BREAK,0,1,,,,,
BRA_00003,BREAK,0,0,,,,,
BRA_00004,BREAK,1,1,,,,,
...
LUN_00001,LUNCH,,0,,1,0,0,0,0
LUN_00002,LUNCH,,0,,0,1,0,1,1
LUN_00003,LUNCH,,1,,0,1,1,0,1
...
DIN_00001,DINNER,,1,,0,1,1,0,0
DIN_00002,DINNER,,0,,0,0,1,0,0
DIN_00003,DINNER,,0,,0,1,1,0,0
...
```

3.9 TimeEvents



(fig. 18: some graphs and CSVs showing some measurements computed using timeEvents)

To compute statistics and record time related events, two classes were implemented:

- **TimeEvent**: represents a single time event, storing its type, and associated information (start time, end time, value) and provides utility methods;
- **TimeEvents**: acts as a container for TimeEvent objects and provides methods to aggregate events and compute statistics.

These classes are used to collect the data required for the computation of the KPIs.

Functions for recording time events are defined in the main and live statistics can be visualized by enabling the livePlots variable in the main simulation (fig. 18).

The following statistics are recorded and computed for every daypart of every day of the week:

- Average customer entrance wait times (minutes)
- Average customer table wait times (minutes)
- Average customer wait times at checkout (minutes)
- Average customer stay time length (minutes)
- Number of customers served (count)
- Average waiters IDLE time (minutes)
- Number of waiter overtime (hours)
- Average number of waiter overtime (hours)
- Average cooks IDLE time (minutes)
- Number of cooks overtime (hours)
- Average number of cooks overtime (hours)
- Percentage of tables usage for every group of tables (%)
- Total earned money (€)
- Total employee costs (€)
- Waiters total salary (€)
- Cooks total salary (€)
- Net profit (€)
- Cumulative net profit (€)

The following statistics are aggregated at other time scales:

- Last customer exit hour for each day
- Waiters total salary for the entire week (€)
- Cooks total salary for the entire week (€)
- Total salaries costs for the entire week (€)
- Total expenses for the entire week (€)
- Total earned at the end of the week (€)
- Net profit at the end of the week (€)

3.10 Scenarios

All the tables that contain parameters also have an integer field named “scenario”. This allows experiments to be run with different configurations of parameter sets. For example, an experiment may use “customers scenario 1”, “employees scenario 3”, “tables scenario 2”, “menu scenario 1”, to evaluate this specific configuration.

A scenario represents a possible configuration of parameters. For example, there could be different customer arrival scenarios such as “moderate inflow”, “festive inflow”, etc. to model different demand conditions.

4. Key System Parameters (KSP) and Key Performance Indicators (KPI):

There are some Key System Parameters that are intended to be explored experimentally:

- Waiter schedule
- Cooks schedule
- Table schedule and availability
- Menu organization, elements and prices (note that changing prices would likely influence customer preferences and popularity weights in ways that are not modeled; therefore, prices should be treated as observed parameters, and not optimized parameters)

Other KSPs are primarily calibration parameters. They are estimated from real observations to make the model as realistic as possible and should not be edited directly:

- Customer arrival rate: this parameter cannot be directly optimized, but it can be estimated from real observations to make the model more realistic. It can also be used to simulate scenarios and evaluate how other parameters should be adjusted.
- Pay time parameters (mean time, standard deviation and the probability that a single customer pays for the whole group);
- Menu reading parameters (mean time, standard deviation, k);
- Menu category correlation matrices, marginal probabilities and dish popularity weights;
- Dish preparation parameters (preparation time mean and standard deviation)
- Eating time parameters (mean time schedule, max time increase, growth parameter)
- Group size (minimum and maximum group size, standard deviation and mean time schedule)
- Fixed costs: a single value that, for simplicity, represents all of the restaurant's operating costs (food, rent, etc.)
- Salaries: standard and overtime hourly wages for the employees. The values are different between waiters and cooks, but for simplicity all employees of a category have the same salary.

The measured Key Performance Indicators are:

- Earned money: computed from the Orders;
- Spent money: computed using salaries, overtimes and fixed costs;
- Customer waiting times: used as an indicator of customer satisfaction;
 - Before entering the restaurant
 - Before table assignment
 - Before checkout
- Checkout time of the last customer of the day;
- Employees overtimes: used to measure an excessive load on the available resources;
- Employees IDLE times: to identify resources in excess;
- Tables utilization: to evaluate the tables allocation and quantities;
- Total customers served: to evaluate the general performance of the restaurant;
- Difference between costs and earned money (net profit).

5. Experimental analysis

Every simulation run spans one week, starting on MONDAY 7:00 AM and ending at 5:00 AM on the next MONDAY. The KPIs are computed for each daypart of every day, allowing the whole week to be evaluated.

Each experiment consists of 100 simulation runs. To analyze the results the mean value of each KPI from all the runs is computed, together with the associated 95% confidence interval.

The CSV files of the results of all simulation runs can be found in `experiments/experiment{n}/`. The computed mean values together with the confidence intervals, standard errors and standard deviations can be found in `experiments/experiment{n}/kpi_summary_outputs`.

The simulated values are computed by anylogic, while the analysis (computing the averages, errors, confidence intervals and the production of plots) is performed by some python scripts (`experiments/kpi.py`, `experiments/plots.py`). To compute the values `kpi.py` should be run first, because `plots.py` generates plots (`experiments/experiment{n}/kpi_summary_outputs/plots/`) with the generated kpis (`experiments/experiment{n}/kpi_summary_outputs/`).

For simplicity, the following experiments will focus on exploring how the model behaves under different employees availability schedules.

5.1 Default parameters

In each experiment only the most relevant parameters are explained explicitly. The parameters, if not included in the explanations, default as follows:

Inside table sizes:	{2,2,2,2,4,4,4,4,4,4,6,6,6,6,6,6,8,8,8,8}
Bar table sizes:	{2,2,2,2,2,4,4,4,6,8}
Outside table sizes:	{2,2,2,2,2,4,4,4,4,8}
Waiters hourly rate:	€12
Cooks hourly rate:	€15
Waiters overtime hourly rate:	€16
Cooks overtime hourly rate:	€19
Cover charge:	€2
Fixed costs:	€1000
Opening time:	7:00 AM
Closing time:	23:00 PM
Min group size:	1
Max group size:	12
Sigma group size:	1
Growth eating time model:	0.35
Max increase eating time model:	0.8
k menu reading model:	0.05
mu menu reading model:	6 min
sigma menu reading model:	2
mu pay time model:	45s
sigma pay time model:	10
p pay time model:	0.7

The restaurant's time-dependent variables can be found in `scripts/restaurant_variables.xlsx`, which contains: the tables selection and priority; the employees schedule; the mean customer eating times; the customer's arrival rates and group sizes.

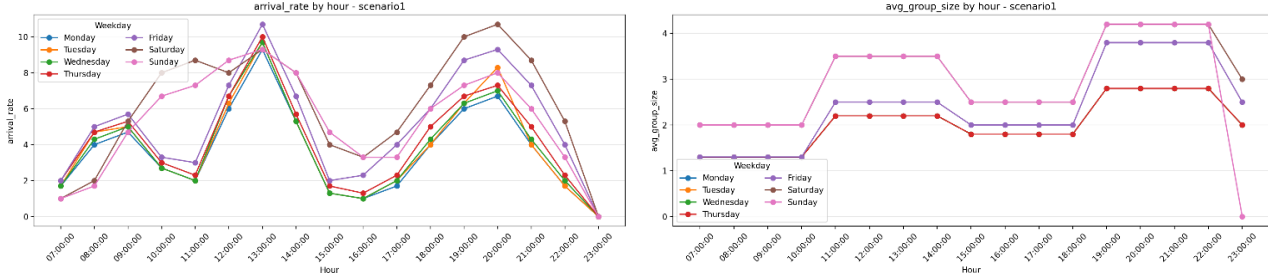
The variables associated with the menu can be found in `scripts/menu.xlsx`, which contains the menu's configuration together with: the dishes mean preparation times and standard deviations; the

courses; the popularity weights; the prices. This file also contains the marginal probabilities for the different categories and the correlation matrices between the categories for the different dayparts.

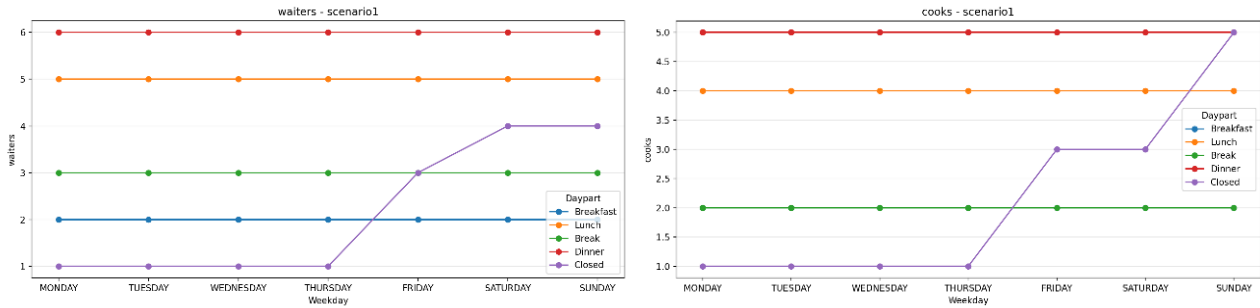
5.2 Experiment 1

5.2.1 Scenario

The first experiment was conducted to understand how the model performs under the default parameters. The relevant parameters are as follows:



(fig. 19, 20: Arrival rates and group sizes in scenario 1)



(fig. 21, 22: Available waiters and cooks in employees scenario 1)

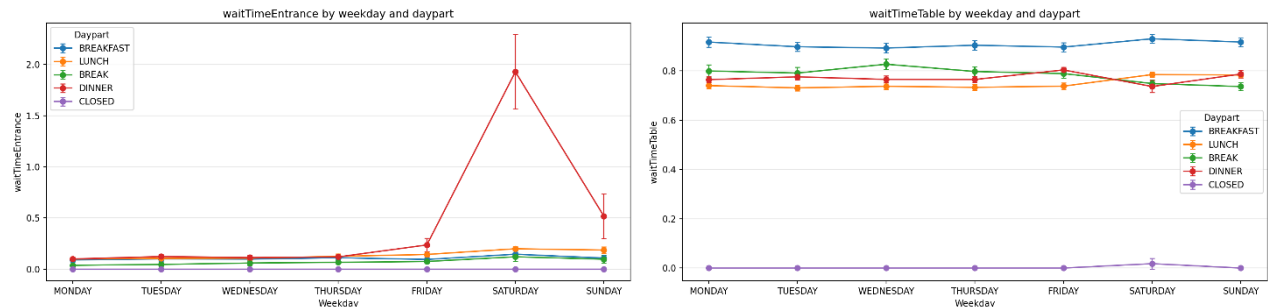
In the arrival rates “scenario 1” (fig.19) the arrival rates follow a wave pattern. Customer arrivals are more frequent during lunch and dinner periods, and the overall demand rises nearing the weekend. A similar behavior can be observed for the average group sizes “scenario 1”(fig. 20).

This configuration will serve as our baseline; the following experiments are aimed to optimize the system’s parameters under this flow of customers.

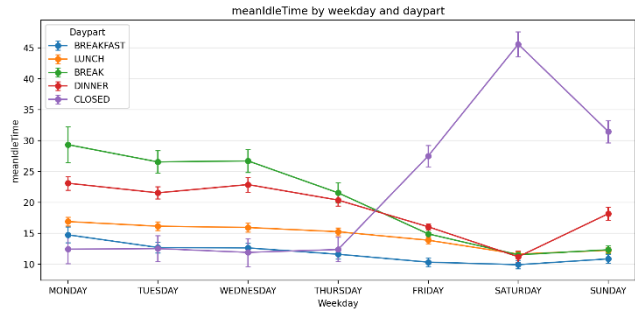
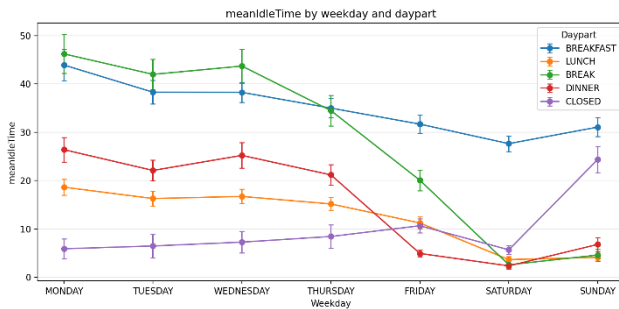
The employees availability “scenario 1” (fig 21, 22) follows a simpler schedule. More employees are assigned during periods of higher demand, especially during weekends when more are allowed to stay overtime to cope with the higher inflow of customers and ensure that the last customers can be served and leave the restaurant.

The purpose of the experiment is to quantify the baseline values of the KPIs. In particular: customer waiting times; last customer leave time; employees IDLE times; costs and net profits.

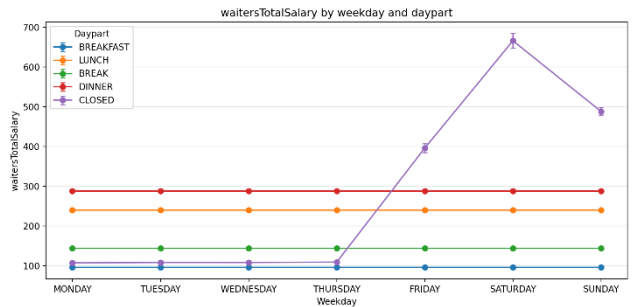
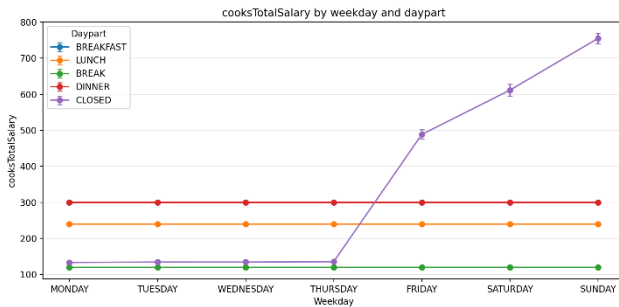
5.2.2 Simulation results



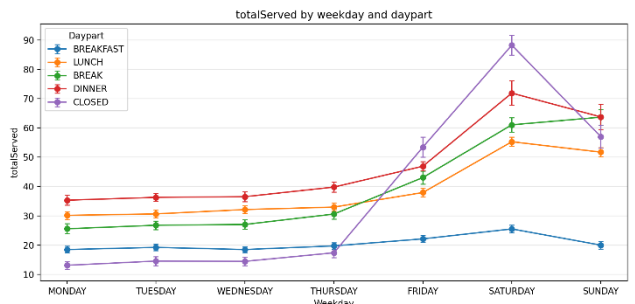
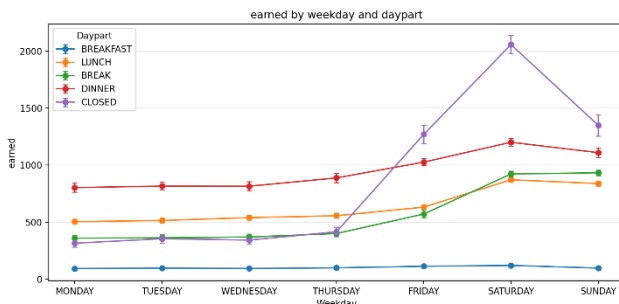
(fig. 23, 24: entrance (left), table assignment (right) wait times (minutes))



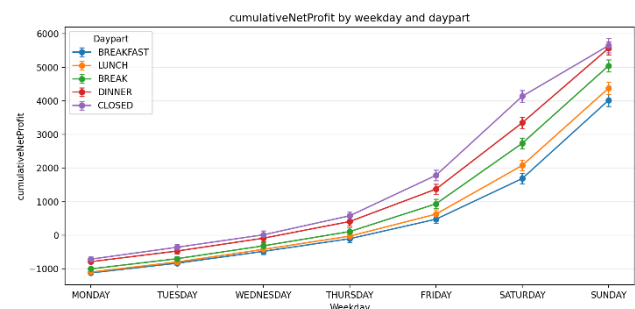
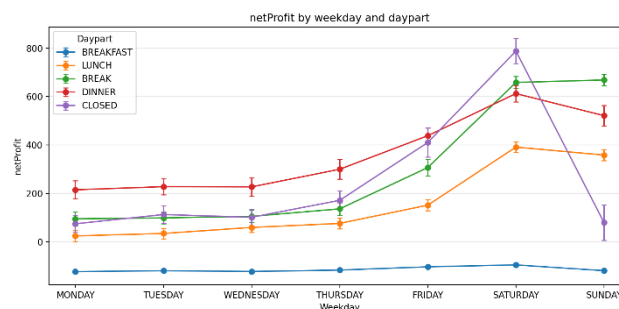
(fig. 25, 26: Cooks (left), Waiters (right) IDLE times (minutes))



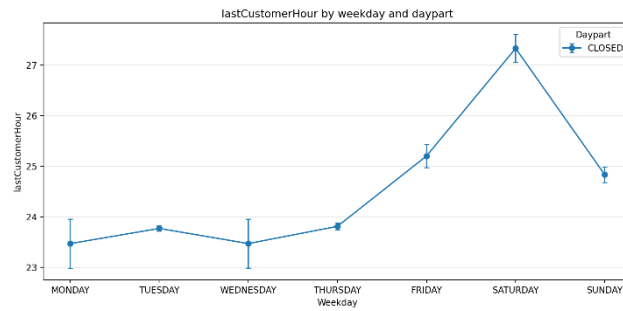
(fig. 27, 28: Cooks (left), Waiters (right) costs (€))



(fig. 29, 30: earned money (left), customers served (right))



(fig. 31, 32: net profit by day/daypart (left), cumulative net profit (right) (€))



(fig. 33: Time of checkout of the last customer of the day¹)

Figures 23, 24 show the customer waiting times at the entrance and during table assignment. The entrance waiting time represents the time spent by the customerGroup before being welcomed by a waiter. The table assignment wait time represents the additional delay experienced when there are no tables of suitable size is currently available.

Both indicators are low throughout the week (the maximum entrance wait time is $1.92 \text{ min} \pm 0.36 \text{ min}$ Saturday at dinnertime), indicating that under the default configuration the parameters don't appear to bottleneck the system at this stage.

Figure 32 shows that the restaurant, at the end of the week, is able to make a profit, with a net profit of $5636.27\text{€} \pm 202.01\text{€}$.

By the results of the simulation, we find some critical aspects:

5.2.3.a) During low demand periods (breakfast, break) cooks and waiters are overstaffed as their IDLE times are high (fig. 25, 26), resulting in lower profits. In particular breakfasts consistently result in a negative net profit (fig. 29, 31);

5.2.3.b) Friday, Saturday and Sunday experience congestion during the dinner period, suggesting an understaffing situation, where resources are unable to keep up with the increased demand (fig. 25, 26). This is also underlined by latest hour a customer exits the restaurant (fig. 33);

5.2.3.c) Every day of the week there are customers that leave the restaurant after the closing time.

To address these issues, we can attempt to modify the model's parameters:

5.2.3.d) To address the first issue we can reduce the number of available employees during breakfast and break periods. This should reduce the IDLE times and employee costs.

5.2.3.e) To mitigate the weekend congestion issue we can increase the number of available employees during lunch and dinner, with the objective to serve more customer groups before the closing time, thus reducing the number of overtime employees and subsequently reducing costs;

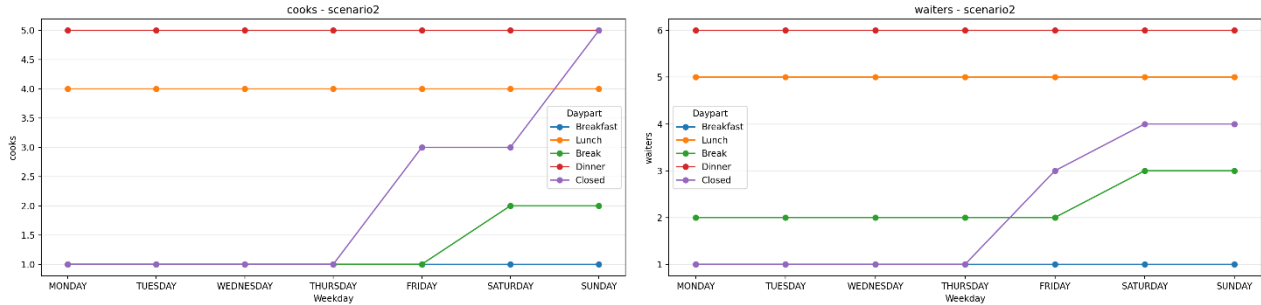
5.2.3.f) The third issue is a structural feature of the model and it does not need to be solved. It does not necessarily indicate overcrowding, but simply reflects that some customers enter the restaurant shortly before closing times and require a sufficient amount of time to eat. Under current modeling assumption the correct behavior is to admit all customers who arrive before the closing time. Consequently no parameters will be modified to mitigate this phenomenon.

¹ Hours later than midnight are represented with values greater than 24 to maintain continuity of the timeline of the business day. For example 25:00 indicates 1:00 AM of the next day.

5.3 Experiment 2

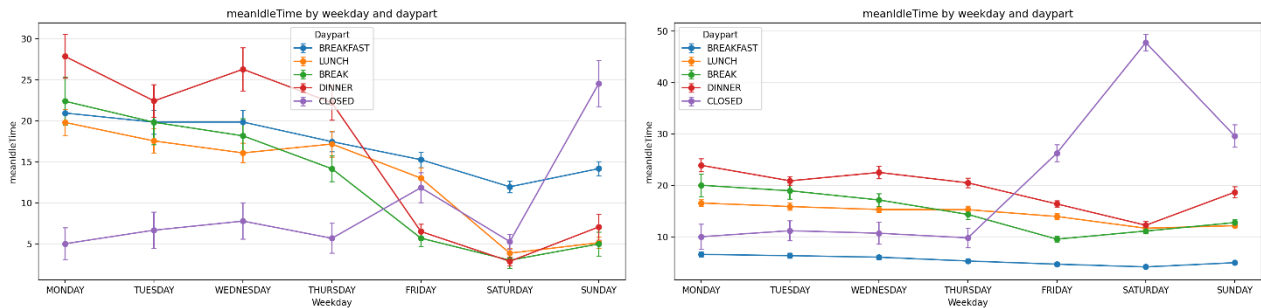
5.3.1 Scenario

This experiment aims to solve the problem 5.2.3.a by applying the solution proposed in 5.2.3.d. For this experiment the employees scenario 2 is used. In this scenario the number of available employees was reduced by one unit except for the weekend (fig. 34, 35).

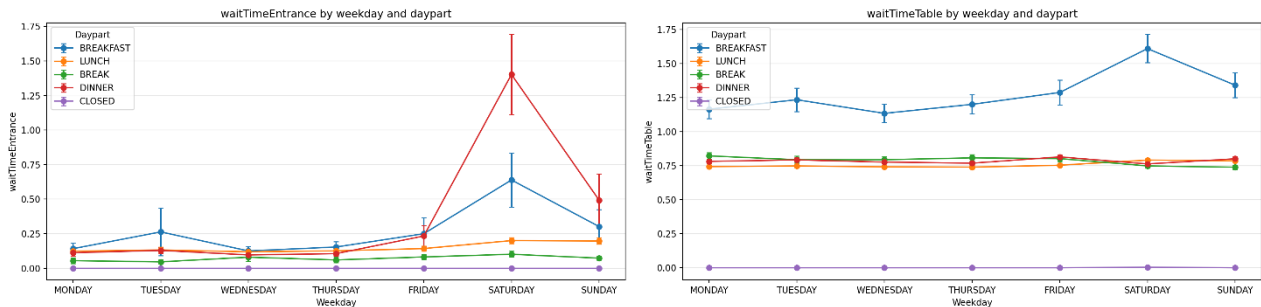


(fig. 34, 35: Available waiters and cooks in employees scenario 2)

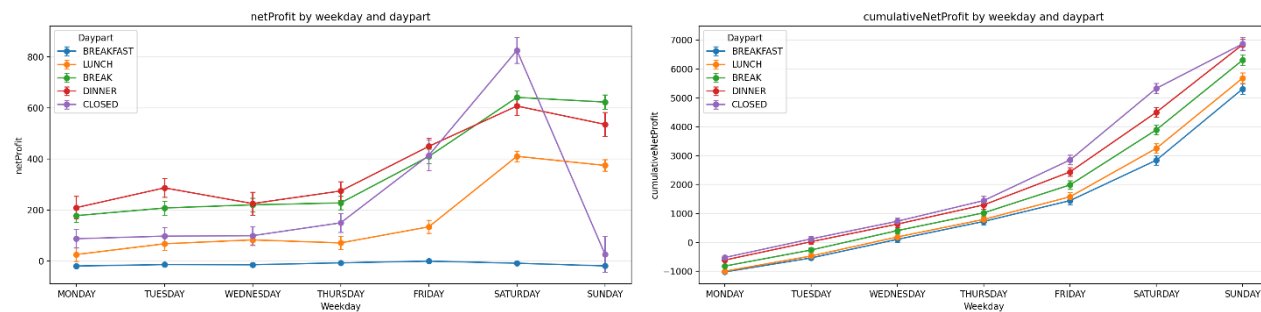
5.3.2 Results



(fig. 36, 37: Cooks (left), Waiters (right) IDLE times (minutes))



(fig. 38, 39: entrance (left), table assignment (right) wait times (minutes))



(fig. 40, 41: net profit by day/daypart (left), cumulative net profit (right) (€))

As shown in the results, employee IDLE times during breakfast and break fell significantly (fig. 36, 37), leading to increased net profits (fig. 40, 41). Although breakfasts still result in a negative profit most days of the week, the magnitude of the loss is substantially reduced and approaches 0.

This experiment, at the end of the simulated week, yields a cumulative net profit of $6867.64 \pm 219.49\text{€}$, representing an increase of $1231,37\text{€}$ compared to the results of experiment 1 ($5636.27\text{€} \pm 202.01\text{€}$). Since the two confidence intervals don't overlap the configuration of experiment 2 produces consistently higher profits.

It should be noted that this cut in the staffing leads to slightly longer waiting times (fig. 38, 39), with effects more pronounced in the weekends, but remain within acceptable limits.

5.4 Experiment 3

5.4.1 Scenario

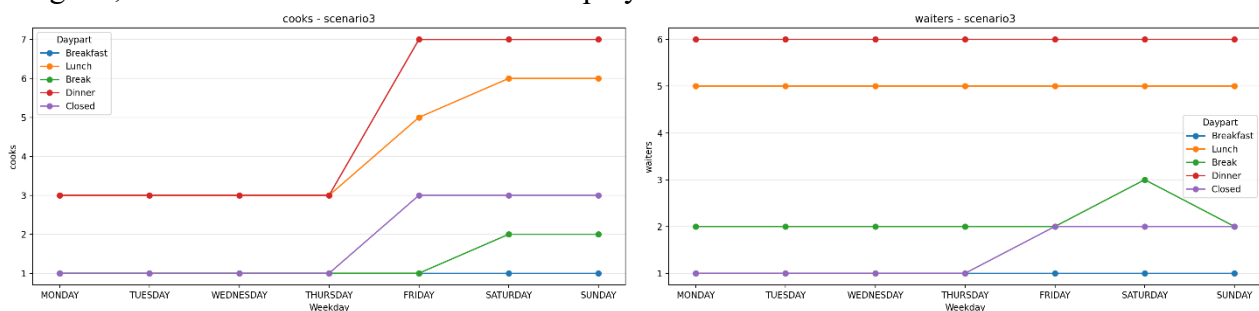
This experiment focuses on redistributing the employees to better match the customers flow addressing the problem described in 5.2.3.b). In particular additional resources are assigned during congested weekend periods, as identified in the previous experiments (fig. 33, 36, 37, 38). They will also be reduced in weekday and overtime periods where high IDLE times were measured (fig. 36, 37).

Additional units are also added during the weekend dinner periods to reduce the number customer groups still being served after closing time. This is expected to reduce the need for overtime employees to further lower costs.

Observing the plots generated in the previous experiments (fig. 36, 37) three observations can be made:

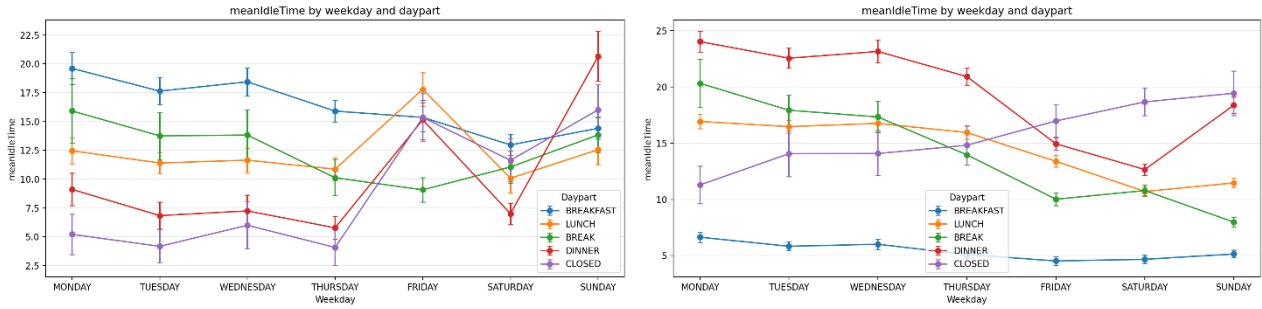
- There are two peaks of IDLE times after closing time (CLOSED) on weekends, indicating that too many cooks and waiters are scheduled overtime;
- In the weekends cooks experience a higher workload relative to waiters. The waiter IDLE chart is flatter, suggesting a more balanced number of resources relative to the flow of customers. This indicates that the kitchen acts as a bottleneck, increasing waiting times (fig. 38).
- On weekdays during lunch and dinner periods, cooks have higher IDLE times and a relatively large number of them are on shift. This suggests that the number of cooks could be reduced without negatively affecting the waiting times.

Fig. 42, 43 show the new distribution of employees in scenario 3.

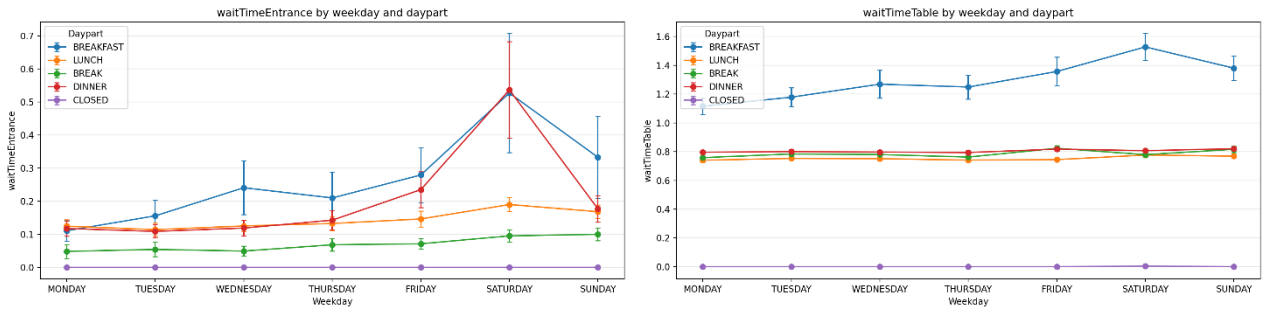


(fig. 42, 43 : Available waiters and cooks in employees scenario 3)

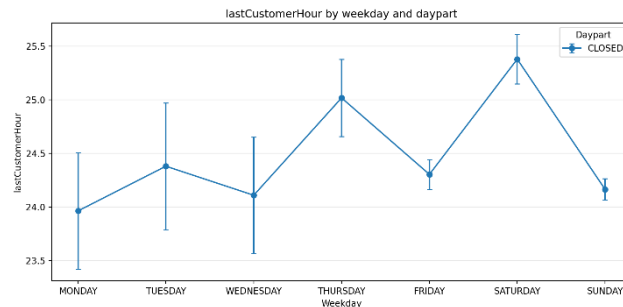
5.4.2 Results



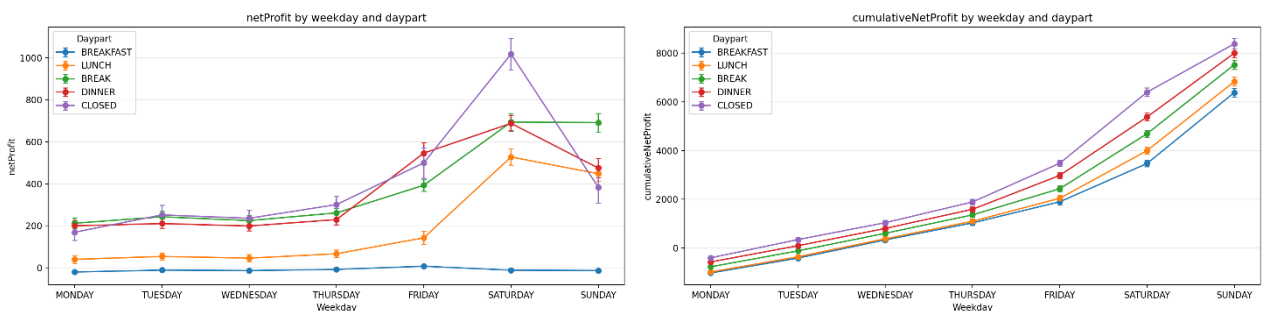
(fig. 44, 45: Cooks (left), Waiters (right) IDLE times (minutes))



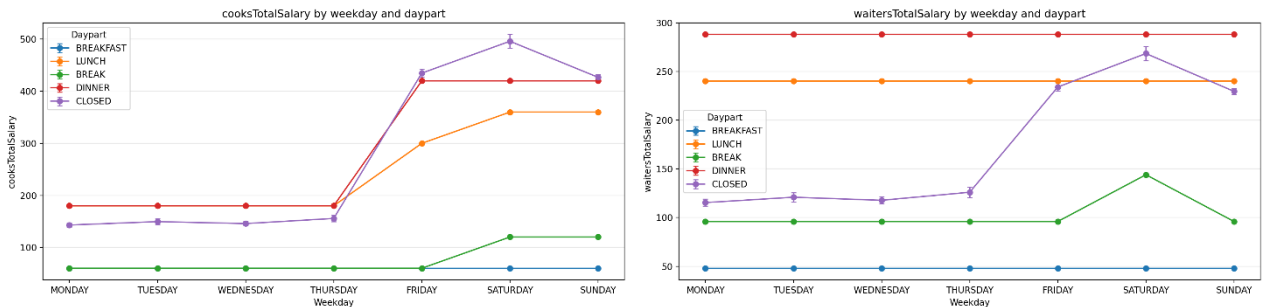
(fig. 46, 47 : entrance (left), table assignation (right) wait times (minutes))



(fig. 48: Time of checkout of the last customer of the day)



(fig. 49, 50: net profit by day/daypart (left), cumulative net profit (right) (€))



(fig. 51, 52: Cooks (left), Waiters (right) costs (€))

Scenario 3 introduces some improvements:

- The reduction of the number of employees effectively decreased IDLE times (fig. 44, 45) in the critical areas identified in section 5.4.1;
- Waiting times did not get worse in periods where resources were removed, indicating that the just introduced congestion can still be absorbed by the available resources. Also, waiting times improved significantly in periods where resources were added, indicating that the previous congestion was resolved. In fig. 48 can be appreciated that the checkout time of the last customer group during the weekend decreased substantially compared to previous experiments (fig. 33). Fig. 46 shows that the entrance waiting times were almost eliminated compared to, for example, fig. 38;
- The reduction of staffing significantly lowered employees related costs (fig. 51, 52), particularly regarding overtimes, compared to previous configurations (fig. 27, 28);
- As a result of these changes, net profits increased even further (fig. 49, 50). Compared to experiment 2 ($6867.64\text{€} \pm 219.49\text{€}$), experiment 3 ($8378.71\text{€} \pm 215.61\text{€}$), yields an additional profit of 1511,07€.

6 Conclusions

This project developed an AnyLogic simulation model of the processes associated with running a restaurant. The model accounts for variations of customer demand, menu choices and resource availability across the day and week. The parameter scenarios and performance indicators allow to spot for inefficiencies and compare alternative configurations.

The first experiment, the baseline, revealed a mismatch between demand and available resources: cooks and waiters experienced long IDLE times during low demand periods (breakfasts, break), and were put under excessive pressure where the demand was higher (weekends). Experiment 2 reduced staffing where needed, lowering IDLE times and the increasing weekly net profit. The third experiment adjusted the schedules more selectively, further reducing staff when it was not needed and adding it during congested periods. This led to even higher net profits and improved customer waiting times.

Among the three configurations the third experiment produced the best results. The mean weekly net profit increased from $5636.27\text{€} \pm 202.01\text{€}$ of experiment 1 to $8378.71\text{€} \pm 215.61\text{€}$ of experiment 3. The new schedule also lowered idle times, employee costs, entrance wait times and improved the checkout time of the last customer group on each day. The results underline the importance that the resource availability is aligned with demand, to improve efficiency and customer waiting times.

These experiments don't establish the optimal parameter configuration, but show that the model responds in a coherent way to the changes in the parameters. The experiments provide a good baseline, but more are needed to evaluate and spot better configurations. Future work could use observed data to calibrate the model, evaluate employee schedules under different demand patterns, and explore alternative table and menu configurations.